COSC 434 – Advanced Web Programming

Lab Session VIII

Building API Endpoints and Asynchronous Interaction in the Recipe App

Spring 2026

Dr. Roaa Soloh
CMS Department

Objective

In this lab session, you will continue the existing **Recipe App** and extend it by introducing **API endpoints** and **asynchronous client–server interaction**.

You have already built the Recipe App using Laravel, Blade, CRUD operations, middleware, and Eloquent relationships.

In this lab, you will move one step further by allowing the application to expose recipe data through JSON responses and interact with that data asynchronously without requiring a full page reload.

Many modern applications need to load, create, update, or delete data dynamically. This session helps you understand how Laravel can support both approaches.

Part One: Conceptual Introduction

Section 1: Understanding APIs in Laravel

What is an API?

An API is a mechanism that allows applications to communicate with each other.

In Laravel, an API often means that instead of returning an HTML page, the application returns **JSON data** that can be used by:

- JavaScript in the browser,
- mobile applications,
- frontend frameworks such as React or Vue,
- external systems.

In your previous Recipe App work, the controller usually returned a Blade view. In this lab, some controller methods will return JSON instead. This is the main shift in thinking.

Example of Blade response:

```
return view('recipes.index', compact('recipes'));
```

Example of JSON response:

```
return response()->json($recipes);
```

How is this different from before?

Before, the controller's job was mainly to prepare data and send it to a Blade page.

Now, the controller can also act as a **data provider**, sending structured JSON that JavaScript or another client can use.

This means:

- before: Laravel returned a full page,
- now: Laravel can return only data.

Why is this useful?

This helps us build applications that are:

- more dynamic,
- easier to extend,
- more suitable for modern frontend interaction,
- better prepared for project work and real-world systems.

RESTful API Basics

The most common HTTP methods used in APIs are:

- **GET**: retrieve data
- **POST**: create new data
- **PUT / PATCH**: update existing data
- **DELETE**: remove data

These methods match the CRUD thinking you already know, but now the communication happens through structured data instead of normal page rendering.

How is this related to what you already know?

Previously, when you created a recipe using a Blade form, the browser sent a request, the controller processed it, and Laravel usually redirected to another page.

Now, the same operation can happen through an API request, where the server returns JSON instead of redirecting to a new page.

So, the logic is still familiar, but the response style is different.

Why Build APIs in Laravel?

APIs are useful because you:

- separate backend logic from presentation,
- make data available to different clients,
- support asynchronous interaction,
- prepare you for modern application architecture.

Benefit for your own projects

If your project later needs:

- filtering without reload,
- live updates,
- integration with JavaScript frameworks,
- mobile access,
- reusable backend endpoints,

then APIs become very important.

That is why this lab is not just an extra topic. It is a practical extension of what you already built.

Section 2: Connecting This Lab to the Existing Recipe App

You are **not** building a new unrelated project.

You are continuing the same Recipe App that already includes:

- recipes CRUD,
- middleware protection,
- one category per recipe,
- multiple tags per recipe.

In other words, relationships are already implemented. In this lab, the API will expose that related data and allow the frontend to work with it asynchronously.

So, in this lab:

- we keep the Recipe App,
- we keep the relationships,
- we keep the existing logic,
- and we add a new communication style.

Part Two: Practical Implementation

Section 3: Preparing the Existing Project

Step 1: Confirm the Recipe App is Working

Before starting, make sure your current Recipe App is functioning correctly.

Check that:

- the recipe list page opens,
- recipe details open,
- categories appear correctly,
- tags appear correctly,
- create and edit features still work,
- middleware behavior still works if you implemented it previously.

Why do we check this first?

Because this lab builds on top of the current project.

If the base project is not stable, then when something fails later, you will not know whether the problem comes from:

- the previous work,
- the API code,
- or the asynchronous frontend code.

Step 2: Keep Using Related Data

Because the Recipe App already uses categories and tags, the API should return that related data too.

Example:

```
$recipes = Recipe::with(['category', 'tags'])->get();
```

Why are we doing this?

Before, the relationships were mainly shown inside Blade views.

Now, we want the API itself to expose that relational structure.

Section 4: Creating the API Controller

Step 1: Generate the Controller

Create a controller for API-style recipe handling:

```
php artisan make:controller API/RecipeController --api
```

Why do we use `--api` here?

Previously, you worked with normal Laravel controllers that return views such as:

- `index`
- `create`
- `edit`

In a normal resource controller, Laravel generates methods like:

- `create()`
- `edit()`

These are useful when you want to return Blade pages that contain forms.

But in this lab, we are focusing on **API endpoints**, not on returning form pages.

That is why `--api` is better here.

It creates only the methods usually needed for APIs:

- `index`
- `store`
- `show`
- `update`
- `destroy`

Why is this better than `--resource` in this lab?

Because:

- `--resource` creates extra methods for form pages,
- `--api` creates a cleaner controller focused on data endpoints only.

So this helps you clearly understand that:

- a web controller often returns pages,
- an API controller usually returns data.

Step 2: Import the Needed Classes

Inside the controller, import:

```
use App\Models\Recipe;
use Illuminate\Http\Request;
```

If needed, also make sure your `Recipe` model already defines:

- `category()`
- `tags()`

Why mention this here?

Because the API controller depends on the model relationships you already created before.

This reinforces an important idea:

the API is not separate from the model structure. It builds on top of it.

Section 5: Creating API Endpoints

Step 1: Add the `index()` Method

This method should return all recipes with related data.

```
public function index()
{
    return response()->json(
        Recipe::with(['category', 'tags'])->get()
    );
}
```

How is this different from before?

Before, listing recipes usually meant returning a Blade page such as `recipes.index`.

Now, listing recipes means returning raw data in JSON format.

Why is this useful?

Because JavaScript can request this data and update the page dynamically without needing a full page reload.

Step 2: Add the `show()` Method

This method returns one recipe.

```
public function show(Recipe $recipe)
{
    return response()->json(
        $recipe->load(['category', 'tags'])
    );
}
```

Why do we still load relationships here?

Because when a single recipe is requested, the client may still need:

- the category,
- the tags.

Step 3: Add the `store()` Method

This method validates and creates a recipe.

```
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'required|string',
        'ingredients' => 'required|string',
        'instructions' => 'required|string',
        'category_id' => 'required|exists:categories,id',
        'tags' => 'nullable|array',
        'tags.*' => 'exists:tags,id',
    ]);

    $recipe = Recipe::create([
        'name' => $validated['name'],
        'description' => $validated['description'],
```

```
            'ingredients' => $validated['ingredients'],
            'instructions' => $validated['instructions'],
            'category_id' => $validated['category_id'],
        ]);

        $recipe->tags()->attach($validated['tags'] ?? []);
        $recipe->load(['category', 'tags']);

        return response()->json([
            'message' => 'Recipe created successfully.',
            'recipe' => $recipe
        ], 201);
    }
```

How is this different from before?

Before, a create form usually submitted data, and Laravel redirected the user to another page. Now, the backend returns a JSON response instead, such as:

- success message,
- created recipe data,
- HTTP status code.

Why is this useful?

Because the frontend can immediately use the returned data to:

- show a success message,
- refresh part of the page,
- update the interface without reloading everything.

That is exactly the kind of behavior you may want in their own projects.

Step 4: Add the `update()` Method

```
    public function update(Request $request, Recipe $recipe)
    {
        $validated = $request->validate([
```

```
            'name' => 'required|string|max:255',
            'description' => 'required|string',
            'ingredients' => 'required|string',
            'instructions' => 'required|string',
            'category_id' => 'required|exists:categories,id',
            'tags' => 'nullable|array',
            'tags.*' => 'exists:tags,id',
        ]);

        $recipe->update([
            'name' => $validated['name'],
            'description' => $validated['description'],
            'ingredients' => $validated['ingredients'],
            'instructions' => $validated['instructions'],
            'category_id' => $validated['category_id'],
        ]);

        $recipe->tags()->sync($validated['tags'] ?? []);
        $recipe->load(['category', 'tags']);

        return response()->json([
            'message' => 'Recipe updated successfully.',
            'recipe' => $recipe
        ]);
    }
```

Why is `sync()` important here?

Before, you learned to use `attach()` when first connecting tags.

During update, we do not want to keep adding duplicate or outdated links.

So `sync()` is more appropriate because it updates the relationship to match the new selected set.

This reminds you that API endpoints still depend on correct relationship logic, not only on returning JSON.

Step 5: Add the `destroy()` Method

```
public function destroy(Recipe $recipe)
{
    $recipe->delete();

    return response()->json([
        'message' => 'Recipe deleted successfully.'
    ]);
}
```

How is this different from before?

Before, deleting often happened through a Blade form and then a redirect.

Now, the backend simply returns a JSON success message, and the frontend decides how to update the page.

Why is this useful?

Because you can remove an item from the screen dynamically without full reload.

Section 6: Registering the Routes

Step 1: Register API Routes

In `routes/api.php`, add:

```
use App\Http\Controllers\API\RecipeController;
Route::apiResource('recipes', RecipeController::class);
```

Why do we use `apiResource()` here?

Previously, you often used `Route::resource()` for normal web routes.

That is useful when the application needs routes such as:

- create form page,
- edit form page.

But here, we are working with API endpoints only.

So `Route::apiResource()` is more suitable because it creates only the API-oriented routes.

Why is this good for you?

It shows you that Laravel provides tools for different architectural needs:

- `resource()` for full web CRUD flow,
- `apiResource()` for API-based CRUD flow.

This is an important distinction for project work.

Step 2: Check the Routes

Run:

```
php artisan route:list
```

Why do we check routes explicitly?

Because you must learn to verify that Laravel registered what you expect.

This is an important debugging habit for their own projects.

Section 7: Testing the API with Postman

Step 1: Start the Server

```
php artisan serve
```

Step 2: Test the Endpoints

Use Postman to test:

Method	Endpoint	Purpose
GET	/api/recipes	List all recipes
GET	/api/recipes/1	Show one recipe
POST	/api/recipes	Create a new recipe
PUT	/api/recipes/1	Update a recipe

DELETE	/api/recipes/1	Delete a recipe

Why do we test with Postman first?

Because before connecting the frontend, we want to confirm that the backend works correctly by itself.

If the request fails in Postman, then the problem is in:

- the controller,
- the route,
- the validation,
- the model logic.

This helps you debug in a more organized way.

Step 3: Example JSON for POST

Go to the **Body** tab in Postman.

Choose:

- **raw**
- **JSON**

Then send something like:

```
{
  "name": "Fruit Salad",
  "description": "A fresh and simple salad.",
  "ingredients": "Apple, banana, orange",
  "instructions": "Cut the fruits and mix them.",
  "category_id": 1,
  "tags": [1, 2]
}
```

Step 4: Add the Needed Header

In **Headers**, add:

Key	Value

Content-Type	application/json
Accept	application/json

Checkpoint:

If the request is valid, you should get:

- `201 Created`
- a JSON response with the new recipe and its relationships

Why is JSON request testing important?

Because you need to understand that in API work, the request body is often structured JSON, not just normal form submission.

That is a major practical difference from earlier labs.

Section 8: Creating a Simple Async Recipe Page

Postman testing is useful, but this lab also aims to show how a page can communicate with the backend asynchronously.

Step 1: Create a New View

Create:

```
resources/views/recipes/api-demo.blade.php
```

Example:

```
@extends('layout')
@section('title', 'Recipe API Demo')
@section('content')
    <h1 class="mb-4">Recipe API Demo</h1>

    <div id="messageBox" class="mb-3"></div>

    <form id="recipeForm" class="mb-4">
```

```
        @csrf

        <input type="text" id="name" class="form-control
mb-2" placeholder="Recipe Name">
        <textarea id="description" class="form-control mb-
2" placeholder="Description"></textarea>
        <textarea id="ingredients" class="form-control mb-
2" placeholder="Ingredients"></textarea>
        <textarea id="instructions" class="form-control mb-
2" placeholder="Instructions"></textarea>

        <select id="category_id" class="form-control mb-2">
            <option value="">Select Category</option>
            @foreach($categories as $category)
                <option value="{{ $category->id }}">{{
$category->name }}</option>
            @endforeach
        </select>

        <div class="mb-3">
            @foreach($tags as $tag)
                <div class="form-check form-check-inline">
                    <input type="checkbox" class="form-
check-input tag-checkbox" value="{{ $tag->id }}">
                    <label class="form-check-label">{{
$tag->name }}</label>
                </div>
            @endforeach
        </div>

        <button type="submit" class="btn btn-primary">Add
```

```
Recipe</button>
    </form>

    <div id="recipeList"></div>
@endsection
```

Why do we still use a Blade page here if this is an API lab?

Because you need to see how APIs can be used from inside their Laravel application too.

This helps you understand that APIs are not only for external tools.

You can also power dynamic behavior inside the same project.

Step 2: Add a Route for This View

In `routes/web.php`:

```
use App\Models\Category;
use App\Models\Tag;

Route::get('/recipes-api-demo', function () {
    return view('recipes.api-demo', [
        'categories' => Category::all(),
        'tags' => Tag::all(),
    ]);
});
```

Checkpoint:

Open the page and confirm the form and list container appear.

Section 9: Loading Recipes Asynchronously

Step 1: Add JavaScript to Fetch Recipes

```
<script>
async function loadRecipes() {
    const response = await fetch('/api/recipes');
```

```
    const recipes = await response.json();

    const recipeList =
document.getElementById('recipeList');
    recipeList.innerHTML = '';

    if (recipes.length === 0) {
        recipeList.innerHTML = '<p>No recipes found.</p>';
        return;
    }

    recipes.forEach(recipe => {
        const categoryName = recipe.category ?
recipe.category.name : 'No Category';
        const tags = recipe.tags.map(tag =>
tag.name).join(', ');

        recipeList.innerHTML += `
            <div class="card mb-3">
                <div class="card-body">
                    <h5>${recipe.name}</h5>
                    <p><strong>Category:</strong>
${categoryName}</p>
                    <p><strong>Tags:</strong> ${tags}</p>
                    <p>${recipe.description}</p>
                    <button class="btn btn-danger btn-sm"
onclick="deleteRecipe(${recipe.id})">Delete</button>
                </div>
            </div>
        `;
    });
```

```
}

document.addEventListener('DOMContentLoaded', loadRecipes);
</script>
```

How is this different from before?

Before, when the page opened, Laravel had already rendered the recipe list on the server.

Now, the page opens first, then JavaScript requests the data afterward.

Why is this useful?

Because the interface becomes more flexible.

You can later add:

- live filtering,
- incremental loading,
- partial refresh,
- interactive dashboards.

Section 10: Adding Recipes Asynchronously

Step 1: Submit the Form Using `fetch()`

```
<script>
document.getElementById('recipeForm').addEventListener('sub
mit', async function(e) {
    e.preventDefault();

    const selectedTags =
Array.from(document.querySelectorAll('.tag-
checkbox:checked'))
        .map(tag => tag.value);

    const payload = {
        name: document.getElementById('name').value,
```

```
        description:
document.getElementById('description').value,
        ingredients:
document.getElementById('ingredients').value,
        instructions:
document.getElementById('instructions').value,
        category_id:
document.getElementById('category_id').value,
        tags: selectedTags
    };

    const response = await fetch('/api/recipes', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'Accept': 'application/json',
            'X-CSRF-TOKEN':
document.querySelector('input[name="_token"]').value
        },
        body: JSON.stringify(payload)
    });

    const result = await response.json();
    const messageBox =
document.getElementById('messageBox');

    if (response.ok) {
        messageBox.innerHTML = `<div class="alert alert-
success">${result.message}</div>`;
        document.getElementById('recipeForm').reset();
        loadRecipes();
```

```
    } else {
        messageBox.innerHTML = `<div class="alert alert-
danger">Failed to create recipe.</div>`;
    }
});
</script>
```

Why are we doing this?

Before, you submitted a form and waited for a full page reload.

Now, the same action happens asynchronously.

This shows clearly that:

- the backend still validates and stores data,
- but the frontend can stay on the same page and update only what is needed.

This is one of the most important transitions in the lab.

Section 11: Deleting Recipes Asynchronously

Step 1: Add a Delete Function

```
<script>
async function deleteRecipe(id) {
    if (!confirm('Are you sure you want to delete this
recipe?')) {
        return;
    }

    const token =
document.querySelector('input[name="_token"]').value;

    const response = await fetch(`/api/recipes/${id}`, {
        method: 'DELETE',
        headers: {
            'Accept': 'application/json',
```

```
                'X-CSRF-TOKEN': token
            }
        });

        const result = await response.json();
        const messageBox =
document.getElementById('messageBox');

        if (response.ok) {
            messageBox.innerHTML = `<div class="alert alert-
success">${result.message}</div>`;
            loadRecipes();
        } else {
            messageBox.innerHTML = `<div class="alert alert-
danger">Failed to delete recipe.</div>`;
        }
    }
</script>
```

Why is this useful for you?

Because it demonstrates that once the backend exposes proper API endpoints, the interface can control how it updates itself.

This is a strong pattern for student projects, especially if you want more interactive pages.

Section 12: Handling Validation Errors

Step 1: Improve Error Display

```
if (!response.ok) {
    if (result.errors) {
        let errorsHtml = '<ul>';
        Object.values(result.errors).forEach(messages => {
            messages.forEach(message => {
                errorsHtml += `<li>${message}</li>`;
```

```
            });
        });
        errorsHtml += '</ul>';

        messageBox.innerHTML = `<div class="alert alert-
danger">${errorsHtml}</div>`;
    } else {
        messageBox.innerHTML = `<div class="alert alert-
danger">Something went wrong.</div>`;
    }
}
```

Why is this important?

Because a good API is not only about success responses.

You need to learn that the backend should also return useful error information, and the frontend should display it clearly.

This improves both debugging and user experience.

Section 13: Optional Protection of API Modifying Actions

If you want to keep management actions protected, you may apply the same middleware concept from the previous lab to the API routes that modify data.

That means you may protect:

- `POST`
- `PUT`
- `DELETE`

while keeping `GET` requests public.